

01. System Architecture & Data Pipeline

This document details the end-to-end data ingestion, preparation, and orchestration pipeline powering the **PredSea** oceanographic forecasting platform.

1. High-Level Data Flow Pipeline

The PredSea automated data pipeline operates on a daily serverless schedule. Upstream global weather and ocean state datasets are retrieved, interpolated across spatial and vertical coordinate systems using parallel processes, fed into coupled numerical solvers, and transformed into optimized NetCDF4 and BigQuery datasets for downstream decision APIs.

```

flowchart TD
    subgraph Upstream Data Sources
        ECMWF["ECMWF Open Data<br/>(IFS Atmospheric Forcing)"]
        CMEMS["Copernicus Marine (CMEMS)<br/>(MED-PHYS 3D Hydrodynamics)"]
    end

    subgraph Data Ingestion & Pre-Processing
        FetchWRF["fetch_ecmwf_forcing.py<br/>(10m Wind, T2, Q2, Rad Fluxes)"]
        FetchCMEMS["fetch_cmems_forcing.py<br/>(u, v, T, S, Zeta Extents)"]
        GILBypass["prepare_croco_forcing.py<br/>(ProcessPoolExecutor Parallel Processing)"]
    end

    subgraph Model Forcing Artifacts
        CROCO_BLK["croco_blk.nc<br/>(Surface Bulk Atmosphere)"]
        CROCO_BRY["croco_bry.nc<br/>(3D Open Boundary Data)"]
        CROCO_CLM["croco_clm.nc<br/>(Climatology Restoring)"]
        CROCO_INI["croco_ini.nc<br/>(Initial Condition State)"]
    end

    subgraph Numerical Compute Engine
        GCPBatch["GCP Batch Spot Instance<br/>(c2d-highcpu-16 / 16 MPI Ranks)"]
        CROCO_Exec["CROCO 2.1.3 Executable<br/>(croco_balearic.exe)"]
    end

    subgraph Storage & Product Ingestion
        GCS_Out["Canonical NetCDF Output<br/>gs://predsea-daily-outputs-test/"]
        ETL_BQ["BigQuery Ingestor<br/>(3,046 Harbors & 127 Routes)"]
        FastAPI["PredSea REST API & Maps<br/>(FastAPI / Deck.gl)"]
    end

    ECMWF --> FetchWRF
    CMEMS --> FetchCMEMS
    FetchWRF --> GILBypass
    FetchCMEMS --> GILBypass

    GILBypass --> CROCO_BLK
    GILBypass --> CROCO_BRY
    GILBypass --> CROCO_CLM
    GILBypass --> CROCO_INI

    CROCO_BLK --> GCPBatch
    CROCO_BRY --> GCPBatch
    CROCO_CLM --> GCPBatch
    CROCO_INI --> GCPBatch

    GCPBatch --> CROCO_Exec
    CROCO_Exec --> GCS_Out
    GCS_Out --> ETL_BQ
    ETL_BQ --> FastAPI

```

2. Atmospheric & Boundary Ingestion Specifications

A. WRF Atmospheric Forcing Ingestion

Atmospheric driving variables are downloaded hourly from ECMWF high-resolution operational models and refined via PredSea's Weather Research and Forecasting (WRF v4.5) 3 km/1 km nested model runs. The required bulk surface forcing parameters include:

Variable Name	Symbol	Units	Physical Description
Zonal Wind Vector	U_{10}	m/s	10 m Eastward wind component
Meridional Wind Vector	V_{10}	m/s	10 m Northward wind component
Air Temperature	T_a	K or °C	2 m Surface air temperature
Specific Humidity	q_a	kg/kg	2 m Specific humidity derived from relative humidity (RH)
Surface Pressure	P_{atm}	Pa	Mean sea level atmospheric pressure
Downward Shortwave Flux	$SW_{\downarrow}$	W/m ²	Solar radiation reaching the sea surface
Downward Longwave Flux	$LW_{\downarrow}$	W/m ²	Atmospheric thermal infrared radiation reaching the sea surface
Precipitation Rate	P_{rate}	kg/m ² /s	Total precipitation for surface freshwater flux calculations

B. CMEMS Hydrodynamic Boundary Ingestion

Open ocean boundary conditions are sourced from the Copernicus Marine Service (CMEMS Mediterranean Physics Analysis and Forecast model). Data fields are extracted in 3D across the full geographic domain:

- **3D Velocity Fields (u, v):** Zonal and meridional oceanic current components across all vertical levels.
 - **3D Temperature (T) & Salinity (S):** Hydrographic state variables resolving pycnoclines and thermoclines.
 - **Sea Surface Height (ζ):** Free-surface barotropic elevation for boundary pressure gradient forcing.
-

3. High-Performance File Preparation Workflow

Translating raw 3D CMEMS and WRF datasets into CROCO-compatible NetCDF input files (croco_blk.nc, croco_bry.nc, croco_clm.nc, croco_ini.nc) was historically a single-threaded bottleneck, requiring **~40 minutes** per 24-hour simulation cycle.

Process-Level GIL Bypass Design

To overcome Python's Global Interpreter Lock (GIL), PredSea's pre-processor scripts/prepare_croco_forcing.py implements process-level concurrency via concurrent.futures.ProcessPoolExecutor.

```
# Architecture snippet: ProcessPoolExecutor GIL bypass for 3D interpolation
from concurrent.futures import ProcessPoolExecutor
import numpy as np

def interpolate_3d_timestep(t_idx, cmems_data, target_croco_grid):
    """
    Independent process worker function interpolating 3D CMEMS variables
    onto CROCO curvilinear rho/u/v points and stretched s-vertical levels.
    """
    # ... spatial SciPy RegularGridInterpolator logic ...
    return interpolated_slice_t

def prepare_croco_forcing_parallel(time_steps, max_workers=16):
    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        futures = [
            executor.submit(interpolate_3d_timestep, t, raw_cmems, target_grid)
            for t in range(time_steps)
        ]
        results = [f.result() for f in futures]
    return assemble_croco_netcdf(results)
```

Pre-Processing Performance Benchmarks

Processing Stage	Legacy Single-Threaded Time	PredSea Parallel Time (c2d-highcpu-16)	Acceleration Factor
Atmospheric Bulk (croco_blk.nc)	8 min 12 sec	0 min 22 sec	22.3x
Open Boundaries (croco_bry.nc)	22 min 45 sec	0 min 58 sec	23.5x
Climatology & Init (croco_clm/ini.nc)	9 min 03 sec	0 min 22 sec	23.9x
Total Pre-Processing Pipeline	40 min 00 sec	1 min 42 sec	23.5x

4. Execution Workflow in `run_marine_simulation.py`

The master orchestration script `simulation/run_marine_simulation.py` executes the following strict sequence:

1. **Validation of Spatial Domain Config:** Parses regional JSON specs (e.g. `balearic_1km.json`) to confirm grid dimensions ($LM = 499$, $MM = 399$, $N = 32$).
2. **Upstream Ingestion Check:** Verifies that WRF surface forcing and CMEMS 3D boundaries exist in GCS staging buckets and contain zero corrupt NaN/Inf values.
3. **Parallel Forcing Assembly:** Triggers `prepare_croco_forcing.py` with multi-core process pools.
4. **Batch Instance Provisioning:** Launches a GCP Batch job on `c2d-highcpu-16` Spot nodes running the compiled CROCO container.
5. **Output Validation & Canonicalization:** Verifies that generated history NetCDF files (`croco_his.nc`) pass physical range gates before writing a durable SUCCESS token to GCS.